ELEC 475: Computer Vision with Deep Learning (Fall 2026)

Lab 2 Post-Lab: Full AlexNet, Normalization Study, & Data Augmentation

Kayne Lee: 20350003

Mark Wang: 20340138

GitHub: https://github.com/kayne-lee/elec-475

October 22, 2026

Architectural Specifications

All models take a 3×64×64 input. Parameter counts are for the 10-class head; the 200-class Tiny ImageNet head adds (in_features + 1) × 190 parameters to the final layer.

Formulas

Output size: $O = \lfloor (I - K + 2P) / S \rfloor + 1$
Parameters: Conv $(K \cdot K \cdot C_{in} + 1) \cdot C_{out}$, Linear $(N_{in} + 1) \cdot N_{out}$, BatchNorm2d $2 \cdot C$
Receptive field: $RF_l = RF_{l-1} + (K_l - 1) \cdot J_{l-1}$, $J_l = J_{l-1} \cdot S_l$, $RF_0 = J_0 = 1$

Base LeNet-5

Two unpadded 5×5 convolutions with Tanh and 2×2 average pooling, followed by three fully connected layers. 338,486 parameters.

Table 1: Base LeNet-5.

Layer	Configuration	Output	Params	RF (px)
Conv1 + Tanh	5×5, s1, p0, 6 filters	6×60×60	456	5
AvgPool1	2×2, s2	6×30×30	0	6
Conv2 + Tanh	5×5, s1, p0, 16 filters	16×26×26	2,416	14
AvgPool2	2×2, s2	16×13×13	0	16
Flatten		2,704	0	
FC1 + Tanh	2,704 → 120	120	324,600	
FC2 + Tanh	120 → 84	84	10,164	
FC3	84 → 10	10	850	

Modernized LeNet

Same filter counts as LeNet-5, with ReLU, padding 2 and 3×3 overlapping max pooling (stride 2). 446,006 parameters.

Table 2: Modernized LeNet.

Layer	Configuration	Output	Params	RF (px)
Conv1 + ReLU	5×5, s1, p2, 6 filters	6×64×64	456	5
MaxPool1	3×3, s2	6×31×31	0	7
Conv2 + ReLU	5×5, s1, p2, 16 filters	16×31×31	2,416	15
MaxPool2	3×3, s2	16×15×15	0	19
Flatten		3,600	0	
FC1 + ReLU	3,600 → 120	120	432,120	
FC2 + ReLU	120 → 84	84	10,164	
FC3	84 → 10	10	850	

AlexNet-64

Five convolutional stages, each Conv → Norm → ReLU, three 3×3 overlapping max pools, and a classifier with Dropout (p = 0.5). Norm is None, LRN (size 5, α = 1e-4, β = 0.75, k = 2, applied after ReLU) or BatchNorm2d. 6,611,402 parameters with BatchNorm2d (2,560 from BN), 6,608,842 without.

Table 3: AlexNet-64.

Layer	Configuration	Output	Params	RF (px)
Conv1 → Norm → ReLU	5×5, s2, p2, 64 filters	64×32×32	4,864	5
MaxPool1	3×3, s2	64×15×15	0	9
Conv2 → Norm → ReLU	5×5, s1, p2, 192 filters	192×15×15	307,392	25
MaxPool2	3×3, s2	192×7×7	0	33
Conv3 → Norm → ReLU	3×3, s1, p1, 384 filters	384×7×7	663,936	49
Conv4 → Norm → ReLU	3×3, s1, p1, 384 filters	384×7×7	1,327,488	65
Conv5 → Norm → ReLU	3×3, s1, p1, 256 filters	256×7×7	884,992	81
MaxPool3	3×3, s2	256×3×3	0	97
Flatten		2,304	0	
Dropout → FC1 → ReLU	p = 0.5, 2,304 → 1,024	1,024	2,360,320	
Dropout → FC2 → ReLU	p = 0.5, 1,024 → 1,024	1,024	1,049,600	
FC3	1,024 → 10	10	10,250	

PyTorch Model Zoo AlexNet

torchvision AlexNet (designed for 224×224 input) with the final layer replaced for 10 classes. No normalization; AdaptiveAvgPool2d(6, 6) before the classifier. 57,044,810 parameters.

Table 4: PyTorch Zoo AlexNet with a 64×64 input.

Layer	Configuration	Output	Params	RF (px)
Conv1 + ReLU	11×11, s4, p2, 64 filters	64×15×15	23,296	11
MaxPool1	3×3, s2	64×7×7	0	19
Conv2 + ReLU	5×5, s1, p2, 192 filters	192×7×7	307,392	51
MaxPool2	3×3, s2	192×3×3	0	67
Conv3 + ReLU	3×3, s1, p1, 384 filters	384×3×3	663,936	99
Conv4 + ReLU	3×3, s1, p1, 256 filters	256×3×3	884,992	131
Conv5 + ReLU	3×3, s1, p1, 256 filters	256×3×3	590,080	163
MaxPool3	3×3, s2	256×1×1	0	195
AdaptiveAvgPool2d	output 6×6	256×6×6	0	
Flatten		9,216	0	
Dropout → FC1 → ReLU	p = 0.5, 9,216 → 4,096	4,096	37,752,832	
Dropout → FC2 → ReLU	p = 0.5, 4,096 → 4,096	4,096	16,781,312	
FC3	4,096 → 10	10	40,970	

Summary

Table 5: Architecture comparison (10-class head).

Model	Conv depth	Params	Conv / FC share	Final RF (px)	Activation / Pooling / Norm
Base LeNet-5	2	0.34 M	0.8% / 99.2%	16	Tanh / Avg 2×2 s2 / None
Modernized LeNet	2	0.45 M	0.6% / 99.4%	19	ReLU / Max 3×3 s2 / None
AlexNet-64	5	6.61 M	48% / 52%	97	ReLU / Max 3×3 s2 / None, LRN or BN
Zoo AlexNet	5	57.04 M	4.3% / 95.7%	195	ReLU / Max 3×3 s2 + AdaptiveAvg / None

Empirical Results & Figures

Master Ablation Tables

Table 6: Architectural configuration and computational complexity (Table 7.3a).

Model	Conv depth	Normalization & Regularization	Data Aug.	Params (M)	FLOPs (M)	Mean Epoch Time (s)	Latency (ms)
1. Base LeNet-5	2	None	None	0.338	7.17	40.6	0.12
2. Modernized LeNet	2	None (ReLU + Overlapping MaxPool)	None	0.446	9.19	44.9	0.15
3. AlexNet (No Norm)	5	Dropout (p = 0.5)	None	6.609	436.70	179.5	1.32
4. AlexNet + LRN	5	LRN + Dropout (p = 0.5)	None	6.609	436.70	226.3	2.22
5. AlexNet + BatchNorm	5	BatchNorm2d + Dropout (p = 0.5)	None	6.611	437.97	180.3	1.24
6. AlexNet + BN + Aug	5	BatchNorm2d + Dropout (p = 0.5)	PyTorch Aug	6.611	437.97	183.3	1.24
7. PyTorch Zoo AlexNet	5	PyTorch Zoo (AdaptiveAvgPool2d)	None	57.045	188.20	108.4	3.54

Epoch time: mean over all epochs on CIFAR-10. All training and latency measurements on CPU (batch size 128 for training, 1 for latency).

Params and FLOPs exceed the handout's reference values (≈6.3 M and ≈250 MFLOPs for AlexNet-64): summing Table 3 layer by layer gives 6.61 M, and FLOPs are counted as 2 × MACs, so the difference is in the reference values.

Table 7: Predictive performance across datasets, best validation epoch (Table 7.3b).

Model	CIFAR-10 Val Acc (%)	CIFAR-10 Val Macro F1 (%)	Tiny ImageNet Val Acc (%)	Tiny ImageNet Val Macro F1 (%)
1. Base LeNet-5	55.67	55.49	11.64	9.82
2. Modernized LeNet	66.71	66.77	20.59	18.53
3. AlexNet (No Norm)	74.03	74.16	17.39	14.98
4. AlexNet + LRN	72.70	72.76	15.72	13.07
5. AlexNet + BatchNorm	78.78	78.86	29.78	28.32
6. AlexNet + BN + Aug	82.74	82.12	29.90	28.38
7. PyTorch Zoo AlexNet	65.96	66.04	20.86	18.62

All runs: Adam, lr = 1e-3, batch size 128, 10 epochs, seed 42, except row 6 CIFAR-10 (15 epochs) and row 7 Tiny ImageNet (lr = 1e-4, see Table 8).

Extended Ablation Exploration

Table 8: Extended ablations, best validation accuracy. Δ = Variant - Baseline (percentage points).

Experiment	Dataset	Baseline	Variant	Baseline Acc (%)	Variant Acc (%)	Δ (pp)
Activation & pooling	CIFAR-10	Base LeNet-5	Modernized LeNet	55.67	66.71	+11.04
Activation & pooling	Tiny ImageNet	Base LeNet-5	Modernized LeNet	11.64	20.59	+8.95
Normalization	CIFAR-10	AlexNet (No Norm)	AlexNet + BN	74.03	78.78	+4.75
Normalization	CIFAR-10	AlexNet (No Norm)	AlexNet + LRN	74.03	72.70	-1.33
Normalization	Tiny ImageNet	AlexNet (No Norm)	AlexNet + BN	17.39	29.78	+12.39
Normalization	Tiny ImageNet	AlexNet (No Norm)	AlexNet + LRN	17.39	15.72	-1.67
Augmentation (15 epochs)	CIFAR-10	AlexNet + BN	AlexNet + BN + Aug	80.44	82.74	+2.30
Augmentation (10 epochs)	Tiny ImageNet	AlexNet + BN	AlexNet + BN + Aug	29.78	29.90	+0.12
Learning rate	Tiny ImageNet	Zoo AlexNet, lr 1e-3	Zoo AlexNet, lr 1e-4	0.57	20.86	+20.29

Generalization Gap

Table 9: Generalization gap of AlexNet + BN on CIFAR-10, final epoch (15). Δ = Train Acc - Val Acc.

Run	Train Acc (%)	Val Acc (%)	Δ (pp)	Best Val Macro F1 (%)
Without augmentation	90.20	80.44	9.76	80.82
With augmentation	84.56	80.24	4.32	82.12

Multi-Seed Evaluation

Table 10: AlexNet + BN on CIFAR-10, 10 epochs, no augmentation, best validation epoch.

Seed	Val Acc (%)	Val Macro F1 (%)
42	78.78	78.86
123	79.25	79.31
999	80.49	80.48
Mean ± Std	79.51 ± 0.88	79.55 ± 0.84

Training Curves

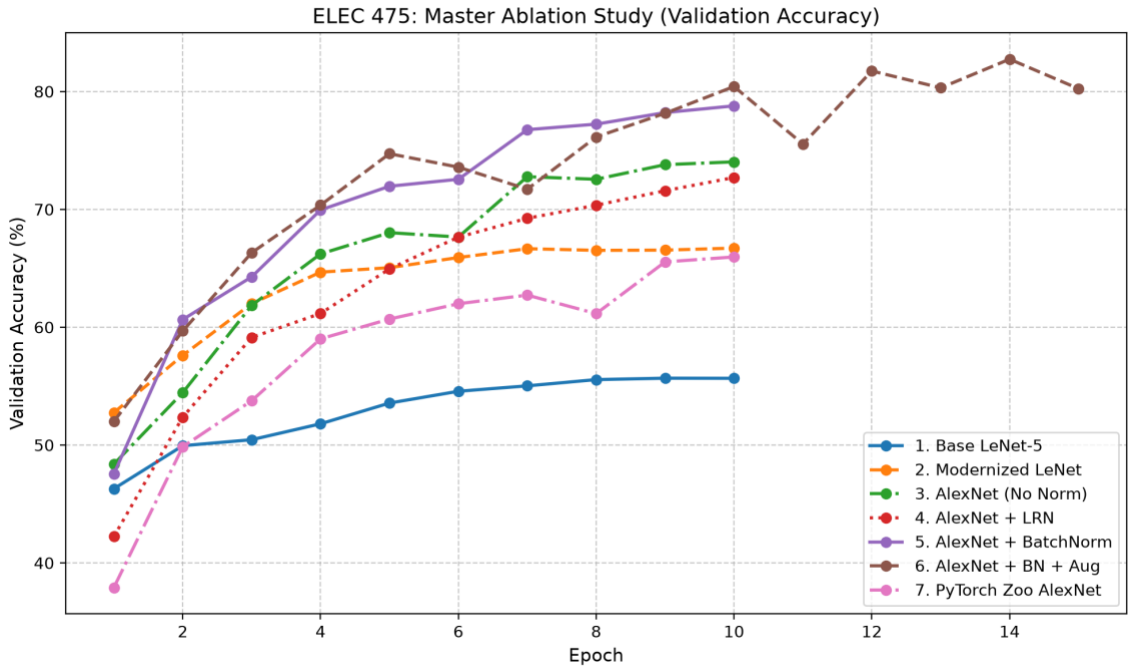


Figure 1: Validation accuracy per epoch for all CIFAR-10 models.

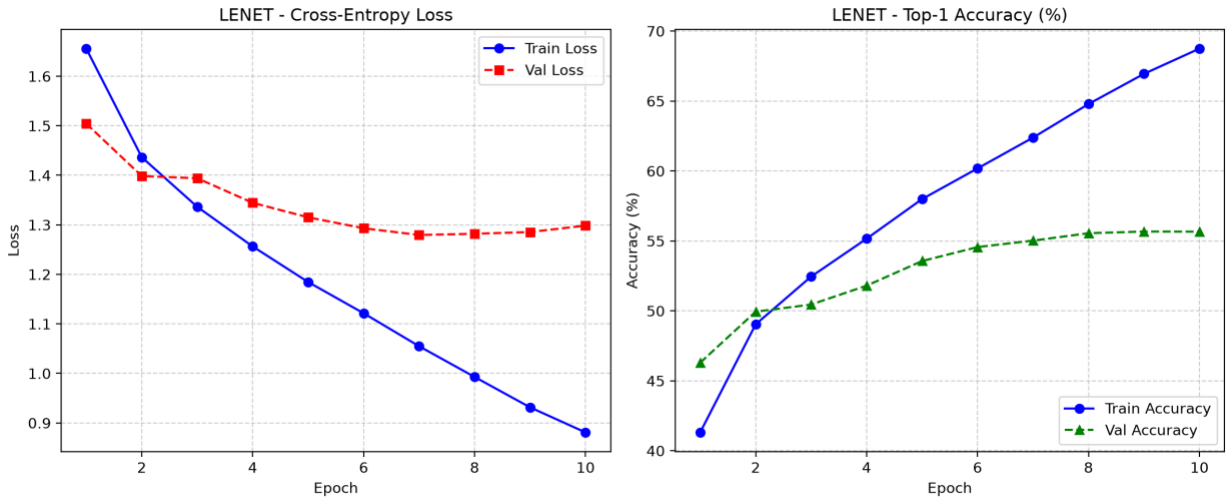


Figure 2: Base LeNet-5, CIFAR-10.

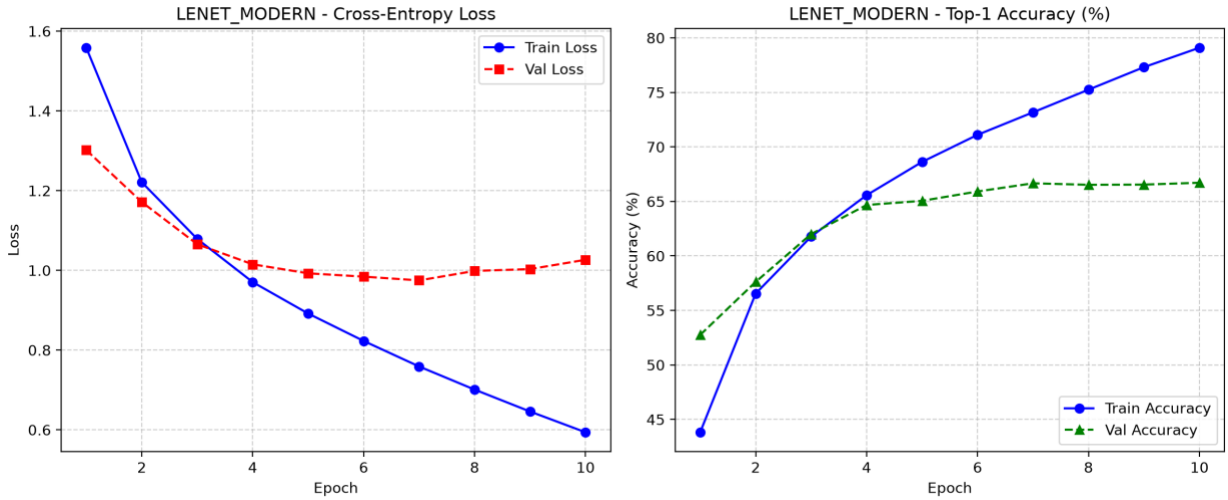


Figure 3: Modernized LeNet, CIFAR-10.

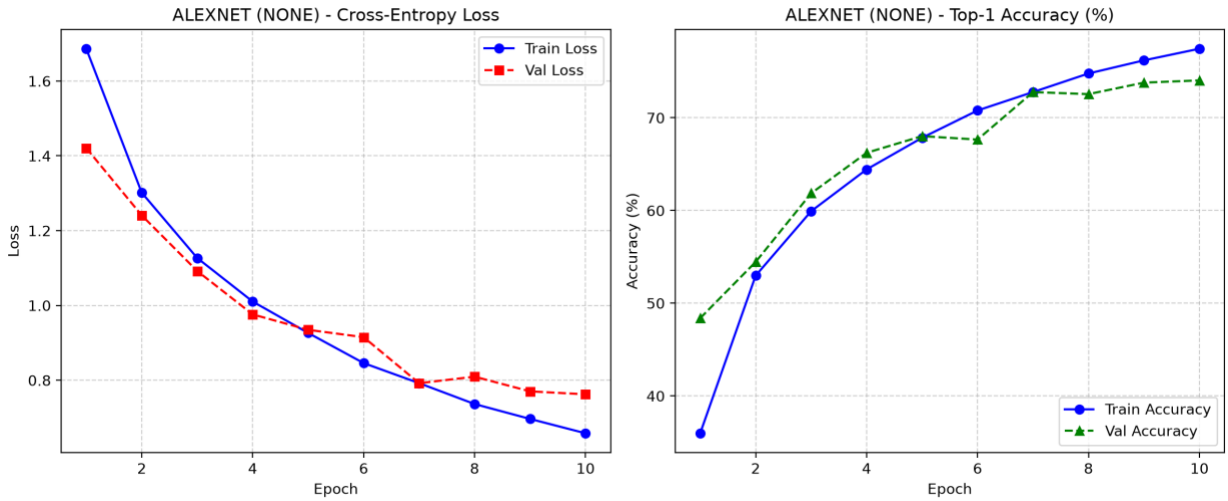


Figure 4: AlexNet (No Norm), CIFAR-10.

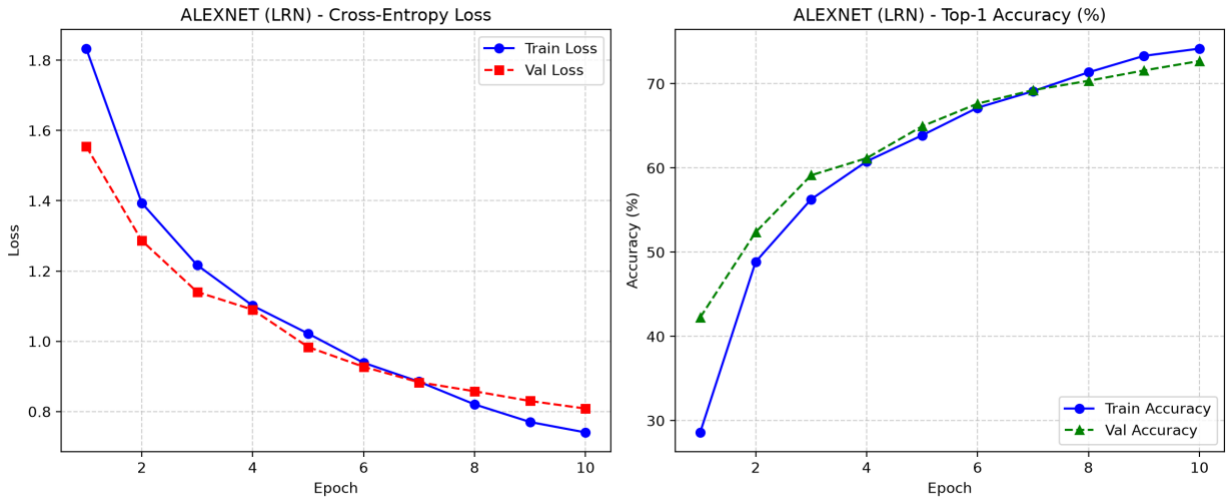


Figure 5: AlexNet + LRN, CIFAR-10.

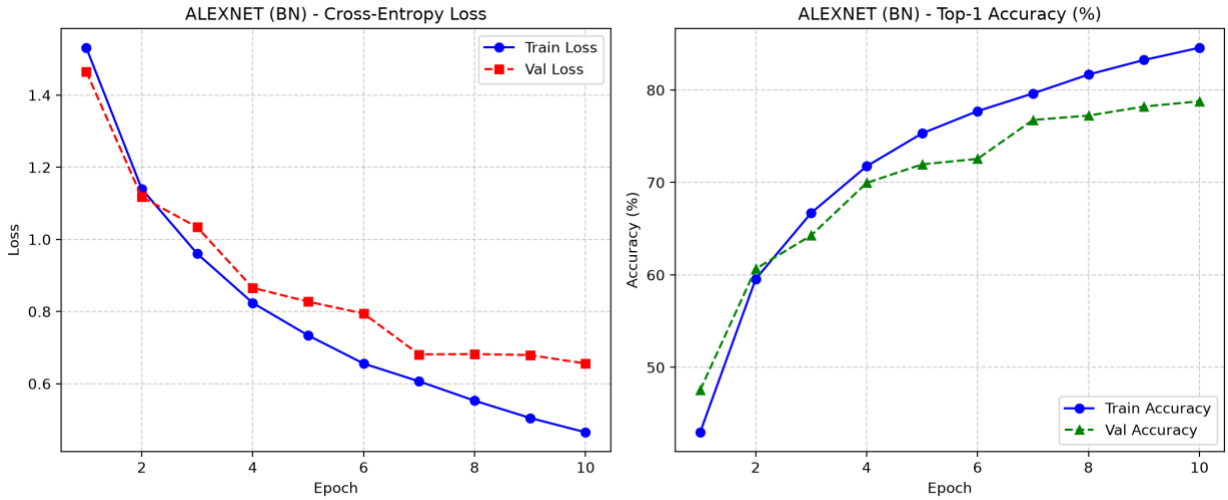


Figure 6: AlexNet + BatchNorm, CIFAR-10.

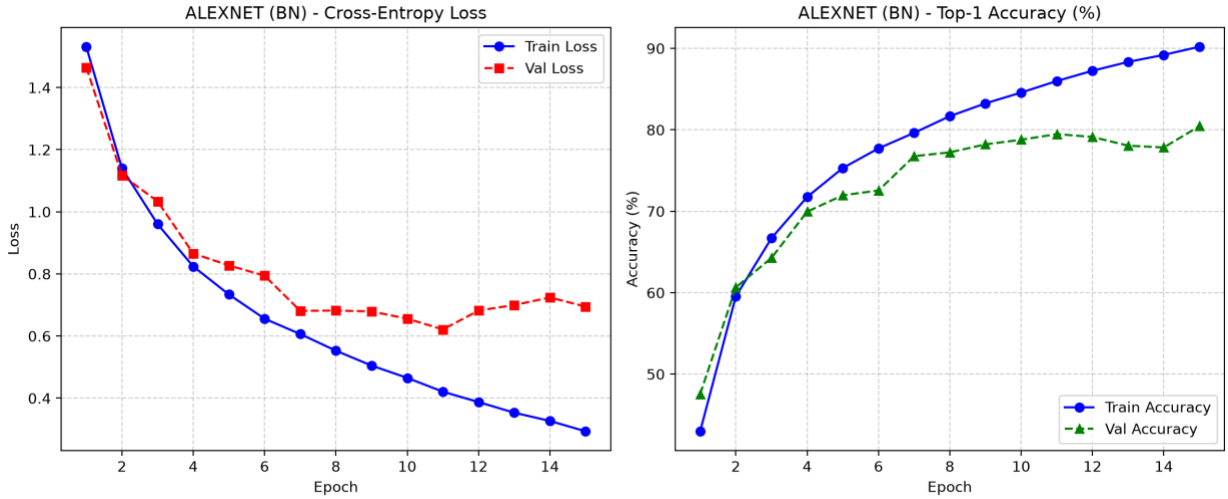


Figure 7: AlexNet + BN without augmentation, CIFAR-10, 15 epochs.

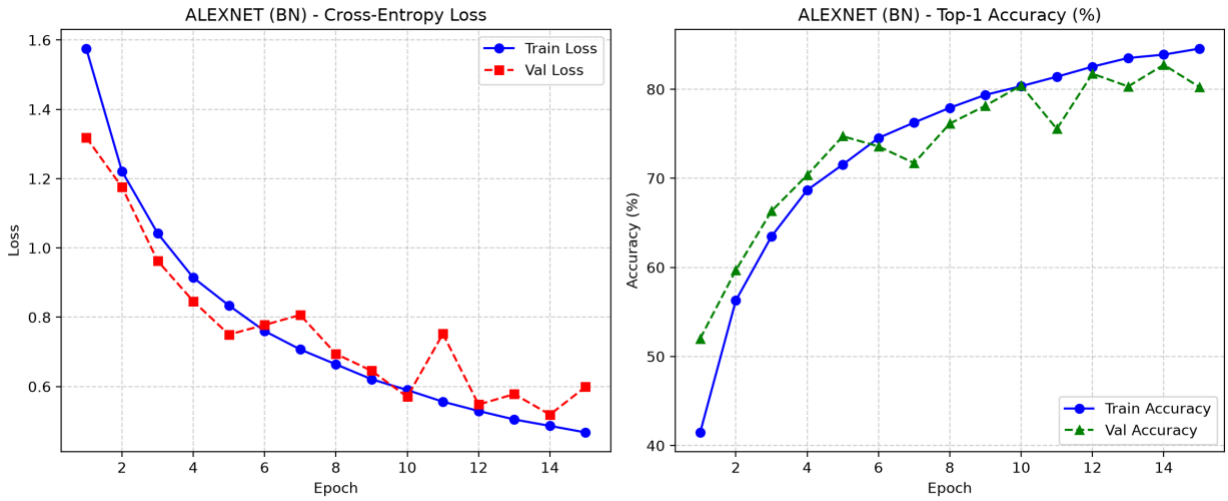


Figure 8: AlexNet + BN with augmentation, CIFAR-10, 15 epochs.

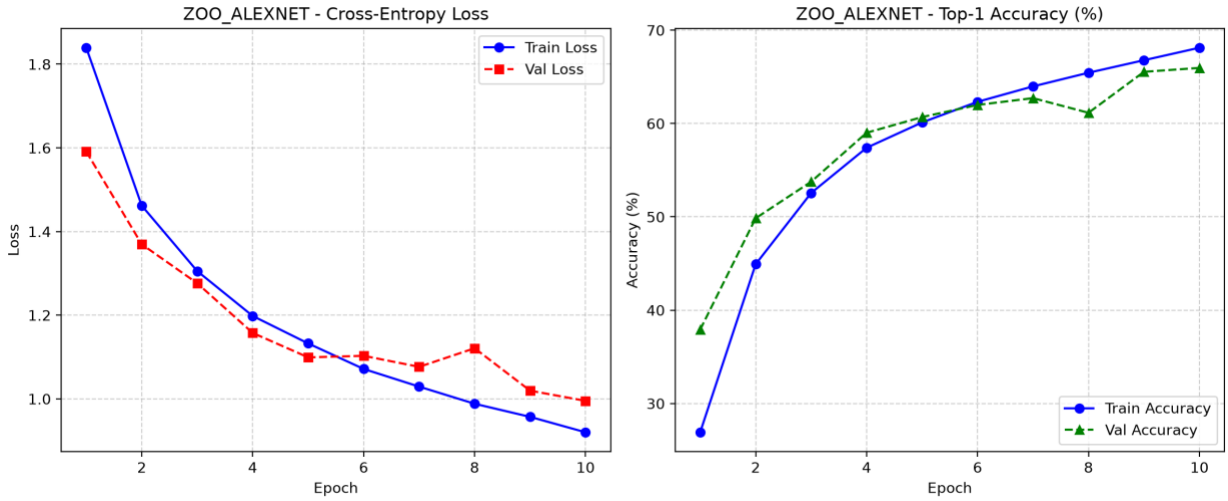


Figure 9: PyTorch Zoo AlexNet, CIFAR-10.

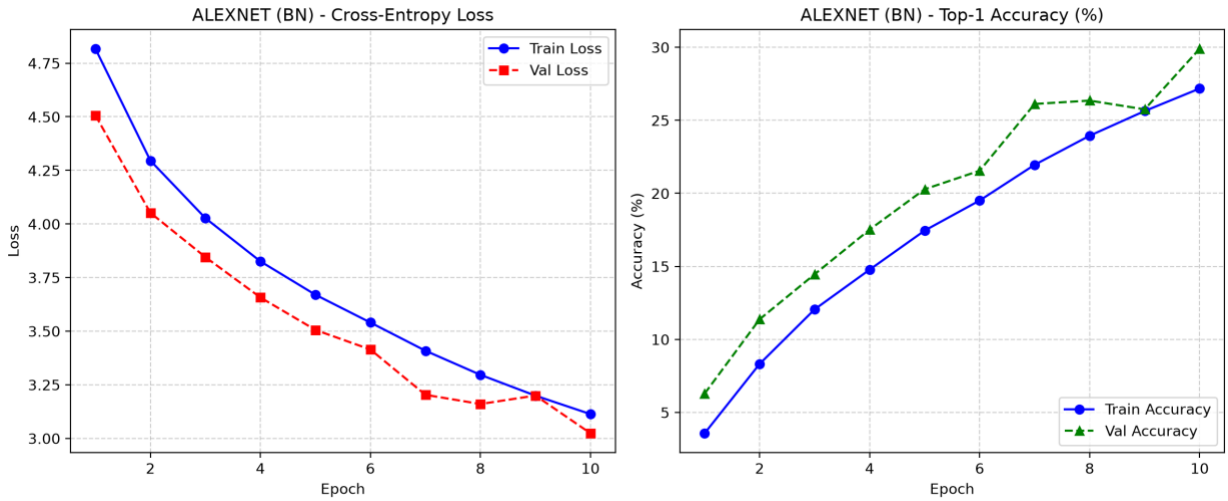


Figure 10: AlexNet + BN + Aug, Tiny ImageNet.

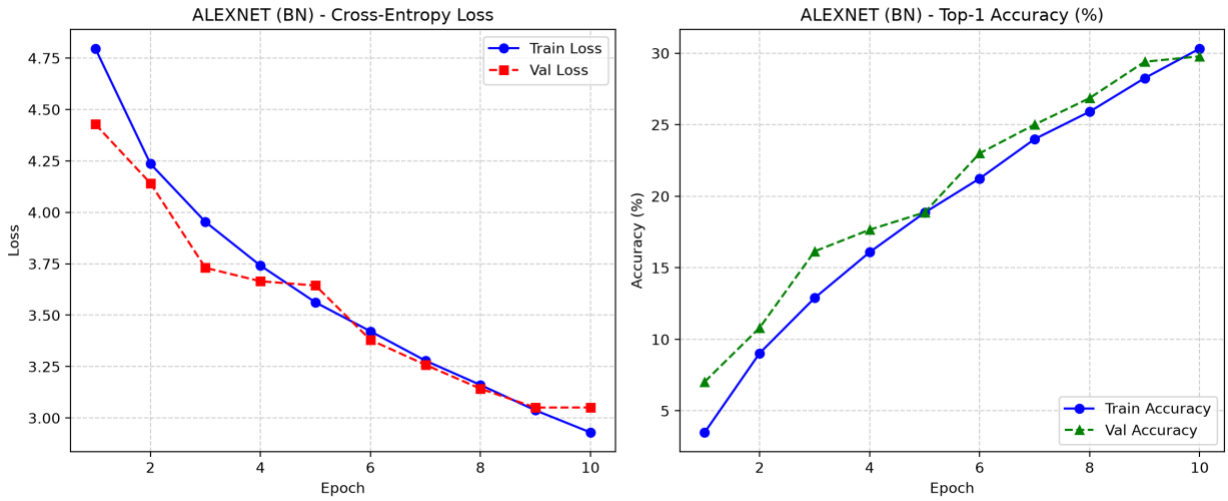


Figure 11: AlexNet + BN, Tiny ImageNet.

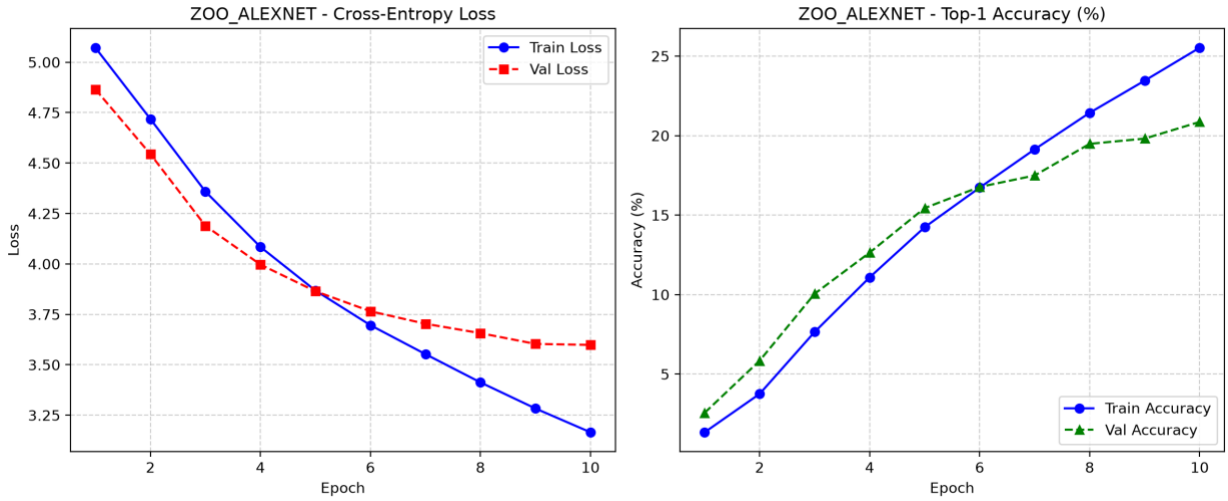


Figure 12: PyTorch Zoo AlexNet, Tiny ImageNet, lr = 1e-4.

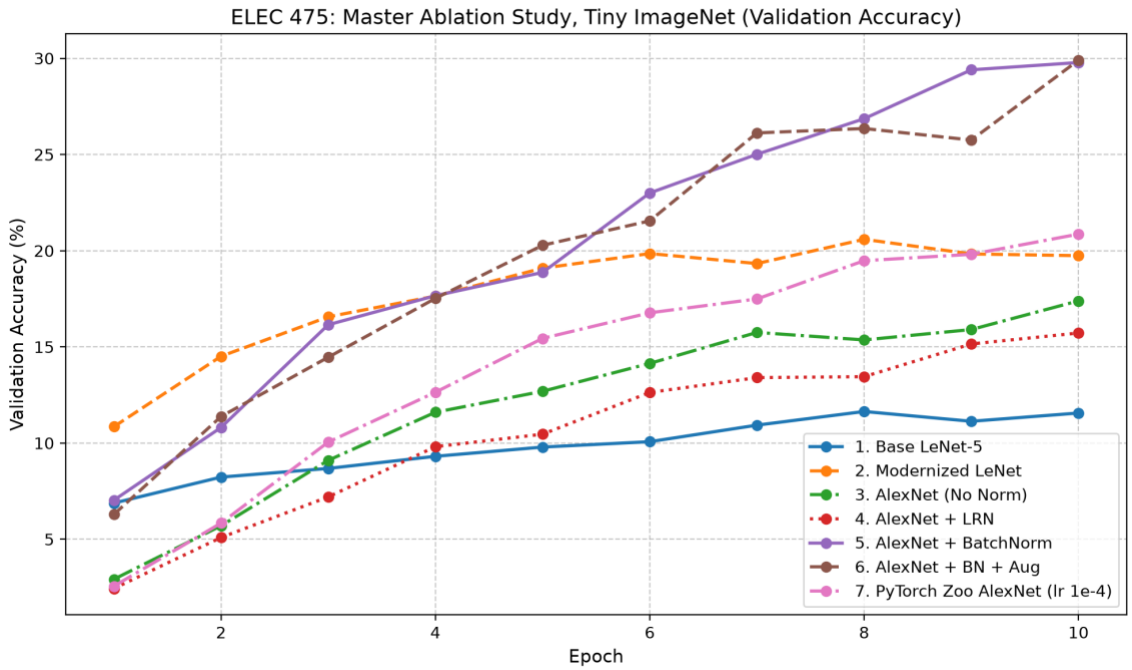


Figure 13: Validation accuracy per epoch for all Tiny ImageNet models.

Model Complexity Benchmark

Output of python compare_models.py --dataset cifar10:

```
(venv) kaynelee@MacBook-Pro-9 lab2_template % python compare_models.py --dataset cifar10
======================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cpu (CPU)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
======================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|--------------|--------------------|
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.12         | 8108.1             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.15         | 6476.0             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 1.32         | 758.4              |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 2.22         | 450.1              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 1.24         | 803.8              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 3.54         | 282.4              |

======================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
======================================================================
(venv) kaynelee@MacBook-Pro-9 lab2_template %
```

Figure 14: Output of compare_models.py on CIFAR-10 (CPU)

Transform Diagnostic

Output of python demo_transforms.py:

```
(venv) kaynelee@MacBook-Pro-9 lab2_template % python demo_transforms.py
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cpu (Host CPU RAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :    5.29 ms  (188,908 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   10.14 ms  ( 98,613 img/s) + Host-to-Device Copy required
- PyTorch (CPU Tensors)   :   47.46 ms  ( 21,069 img/s)
================================================================================
(venv) kaynelee@MacBook-Pro-9 lab2_template %
```

Figure 15: Output of demo_transforms.py.

Exploratory Data Analysis

Table 11: Dataset summary and per-channel normalization statistics.

Dataset	Classes	Train images	Val images	Train per class	Imbalance ratio	Mean (R, G, B)	Std (R, G, B)
CIFAR-10	10	50,000	10,000	5,000	1.00	0.491, 0.482, 0.447	0.247, 0.244, 0.262
Tiny ImageNet	200	100,000	10,000	500	1.00	0.480, 0.448, 0.398	0.276, 0.269, 0.282

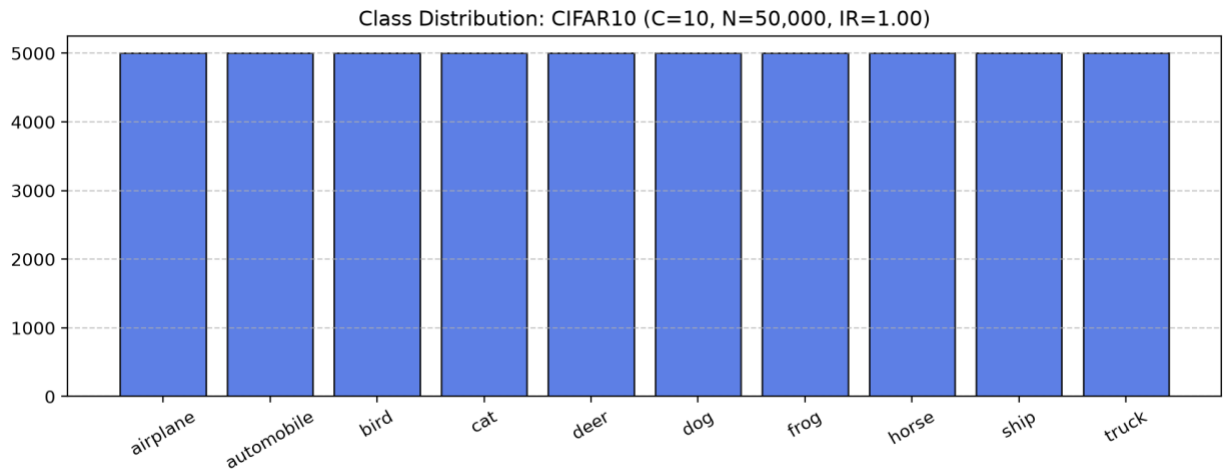


Figure 16: CIFAR-10 class distribution.

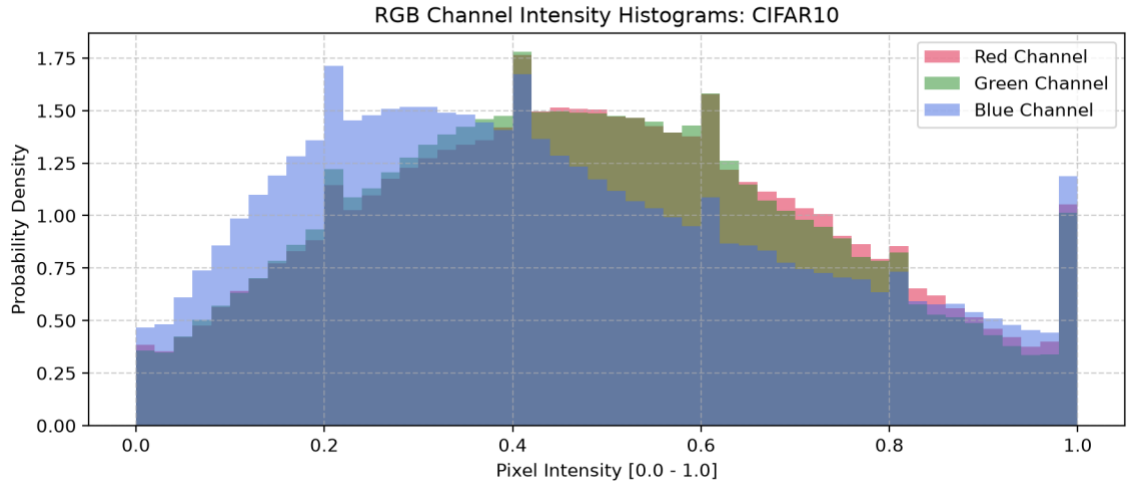


Figure 17: CIFAR-10 RGB channel distributions.

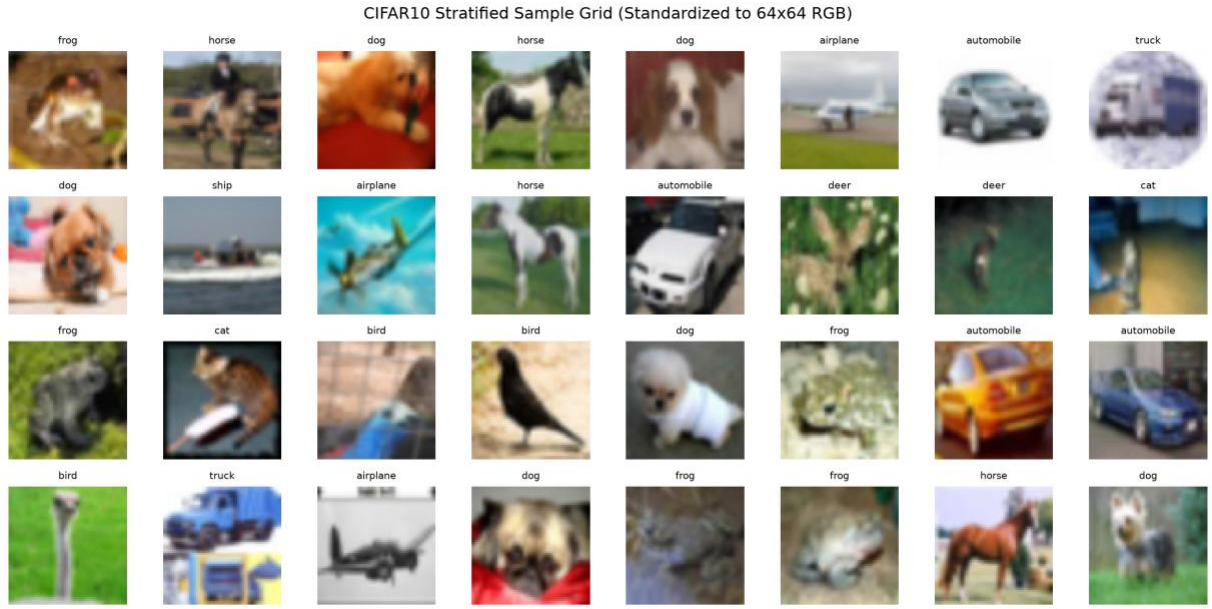


Figure 18: CIFAR-10 sample images.

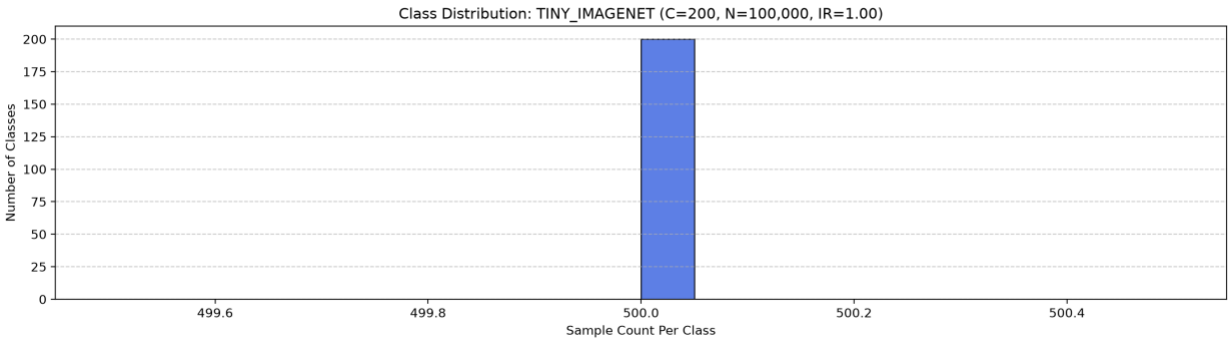


Figure 19: Tiny ImageNet class distribution.

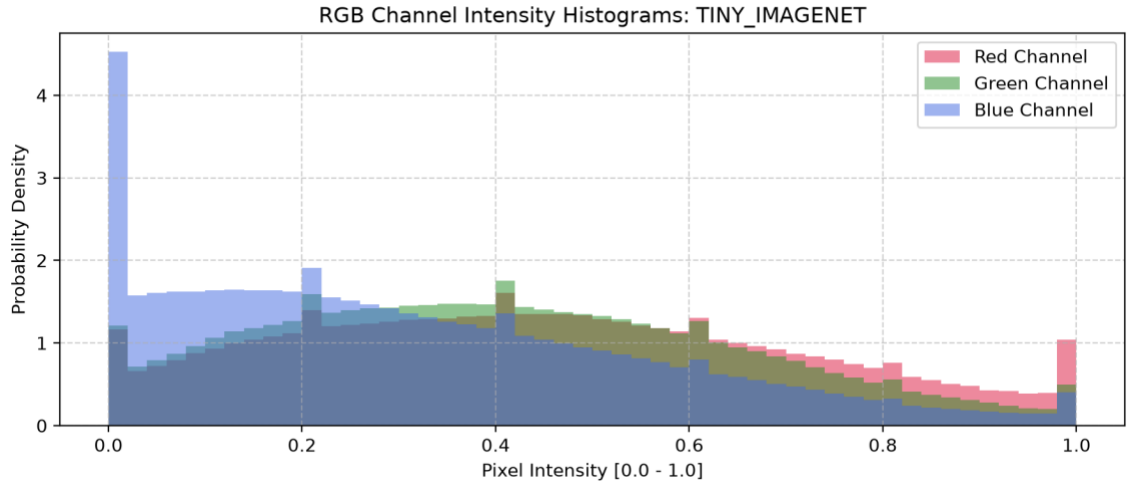


Figure 20: Tiny ImageNet RGB channel distributions.



Figure 21: Tiny ImageNet sample images.

Technical Analysis & Discussion

Normalization Study

Performance improvement: Yes. BN reached 78.78% accuracy and 78.86% macro F1, against 74.03% / 74.16% with no normalization and 72.70% / 72.76% with LRN. BN rescales each channel's pre-activations to zero mean and unit variance over the mini-batch, which reduces internal covariate shift between layers and smooths the loss landscape, so Adam can make larger effective steps without diverging.

Computational overhead: Yes for LRN, negligible for BN. BN adds 0.3% FLOPs and 0.8 s per epoch; LRN adds no counted FLOPs but increases epoch time by 26% (226.3 s vs. 179.5 s) and latency by 68% (2.22 ms vs. 1.32 ms), because its cross-channel window is memory-bound. LRN costs time and lowers accuracy, while BN improves accuracy by 4.75 pp at almost no cost.

Convergence stability: All three training losses decrease almost monotonically (Figures 4 to 6). BN converges fastest and to the lowest validation loss (0.66, vs. 0.76 for no normalization and 0.81 for LRN); LRN is slowest. The BN result is stable across seeds (79.51 ± 0.88%, Table 10).

Data Augmentation

Generalization gap reduction: Yes. Δ fell from 9.76 pp (90.20% train, 80.44% val) to 4.32 pp (84.56% train, 80.24% val) at epoch 15 (Table 9).

Validation metric improvement: Yes. Best accuracy rose from 80.44% to 82.74% and macro F1 from 80.82% to 82.12%. Random crops, flips and colour jitter keep the label but change position, orientation and lighting, so the filters must learn features of the object rather than memorize training images. Without augmentation, validation loss rises after epoch 11 (Figure 7); with augmentation it reaches its minimum (0.52) at epoch 14 (Figure 8).

Tiny ImageNet Benchmark

Above random chance: Yes. 29.90% accuracy, about 60 times the 0.5% chance level.

Cross-dataset comparison: 29.90% / 28.38% F1 on Tiny ImageNet vs. 82.74% / 82.12% on CIFAR-10. Tiny ImageNet has 200 classes instead of 10, only 500 training images per class instead of 5,000, and many visually similar fine-grained classes.

Overfitting trajectory: No. Tiny ImageNet validation loss is still decreasing at epoch 10 (4.51 to 3.02, Figure 10), and training accuracy (27.18%) is below validation accuracy (29.90%) because dropout and augmentation are active only during training. The model is underfitting, whereas CIFAR-10 without augmentation started to overfit.

Model Variations

LeNet-5 to Modernized LeNet: +11.04 pp on CIFAR-10. ReLU avoids Tanh saturation and overlapping max pooling keeps the strongest responses and larger feature maps.

LeNet to AlexNet-64: Five stages and a 97 px receptive field covering the whole input, compared with 16 to 19 px for LeNet, allow features that describe entire objects.

Zoo AlexNet vs. AlexNet-64: 65.96% vs. 74.03% to 78.78% on CIFAR-10 despite 8.6 times more parameters. Its 11×11 stride-4 first layer and pooling reduce a 64×64 input to 1×1 before the classifier, so AdaptiveAvgPool2d only repeats 256 values. Without normalization it also failed to train on Tiny ImageNet at lr = 1e-3 (0.57%, Table 8).

Conclusion

BatchNorm had the largest effect: +4.75 pp on CIFAR-10 and +12.39 pp on Tiny ImageNet over no normalization, at almost no extra cost, while LRN was slower and less accurate. ReLU and overlapping max pooling added 11.04 pp to LeNet. Data augmentation cut the generalization gap from 9.76 to 4.32 pp and gave the best CIFAR-10 result (82.74%). The Zoo AlexNet, despite 8.6 times more parameters, underperformed because its design for 224×224 inputs collapses a 64×64 image to 1×1. Tiny ImageNet models were still underfitting after 10 epochs, so longer training would likely improve them.